

Data Structures and Algorithms

DSA

Table of content

1. Introduction to data structures and algorithms
2. Asymptotic notation
3. Time complexity
4. Space complexity
5. Mathematical concepts
6. Bit in computers
7. Bitwise operators
8. Recursion
9. Arrays

Introduction to Data structures and algorithms

Data Structure is a way of collecting and organizing data to perform operations on these data effectively. Data Structures is about rendering data elements in terms of some relationship for better organization and storage. For example, we have some data which has, player's name "Virat" and age 26. Here "Virat" is of String data type, and 26 is of integer data type.

We can organize this data as a Player record, with both player's names and ages. Now we can collect and store player's records in a file or database as a data structure. For example, "Dhoni" 30, "Gambhir" 31, "Sehwag" 33

If you are aware of Object-Oriented programming concepts, then a class also does the same thing. It collects the different types of data under one single entity. The only difference being, data structures provide techniques to access and manipulate data efficiently.

In simple language, Data Structures are structures programmed to store ordered data to perform various operations easily. It represents the knowledge

of data to be organized in memory. Therefore, it should be designed and implemented in such a way that it reduces complexity and increases efficiency.

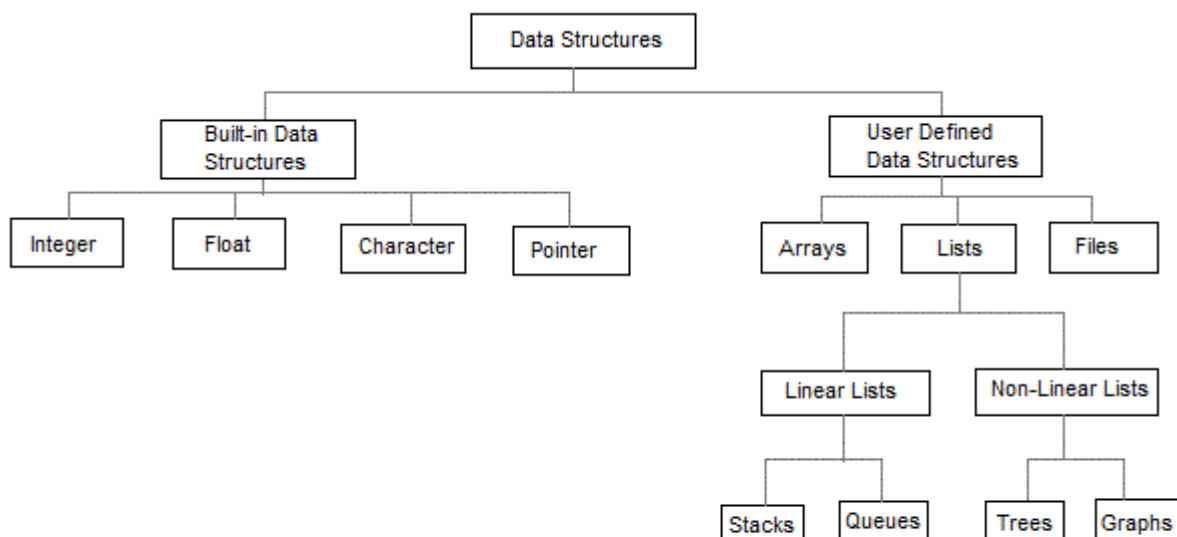
Basic types of Data Structures

As discussed above, anything that can store data can be called a data structure. Hence Integer, Float, Boolean, Char, etc. All are data structures. Hence, they are known as Primitive Data Structures.

Then we also have some complex Data Structures, which are used to store large and connected data. Some examples of Abstract Data Structure are :

- Linked List
- Tree
- Graph
- Stack, Queue, etc.

All these data structures allow us to perform different operations on data. We select these data structures based on which type of operation is required. We will look into these data structures in more detail in our later lessons.



INTRODUCTION TO DATA STRUCTURES

The data structures can also be classified based on the following characteristics:

Characteristic	Description
Linear	In Linear data structures, the data items are arranged in a linear sequence. Example: Array
Non-Linear	In Non-Linear data structures, the data items are not in sequence. Example: Tree, Graph
Homogeneous	Inhomogeneous data structures, all the elements are of the same type. Example: Array
Non-Homogeneous	In a Non-Homogeneous data structure, the elements may or may not be of the same type. Example: Structures
Static	Static data structures are those whose sizes and structures associated memory locations are fixed at compile time. Example: Array

Dynamic	Dynamic structures are those which expand or shrink depending upon the program's need and its execution. Also, their associated memory locations changes. Example: Linked List created using pointers
---------	---

What is an Algorithm?

An algorithm is a finite set of instructions or logic written to accomplish a certain predefined task. The algorithm is not the complete code or program. Instead, it is just the core logic(solution) of a problem, which can be expressed either as an informal high-level description as pseudocode or using a flowchart.

Every algorithm must satisfy the following properties:

1. Input- we should supply 0 or more inputs externally to the algorithm.
2. Output- we should obtain at least one output.
3. Definiteness- Every step of the algorithm should be clear and well-defined.
4. Finiteness- The algorithm should have a finite number of steps.
5. Correctness- Every step of the algorithm must generate the correct output.

An algorithm is efficient and fast if it takes less time to execute and consumes less memory space. The performance of an algorithm is measured based on the following properties :

1. Time Complexity
2. Space Complexity

Space Complexity

It's the amount of memory space required by the algorithm during its execution. We must take space complexity seriously for multi-user systems and in situations where limited memory is available.

An algorithm generally requires space for the following components :

- **Instruction Space:** It's the space required to store the executable version of the program. This space is fixed but varies depending upon the number of lines of code in the program.
- **Data Space:** It's the space required to store all the constants and variables(including temporary variables) values.
- **Environment Space:** It's the space required to store the environment information needed to resume the suspended function.

To learn about Space Complexity in detail, jump to the Space Complexity tutorial.

Time Complexity

Time Complexity is a way to represent the amount of time required by the program to run till its completion. It's generally a good practice to keep the required time minimum so that our algorithm completes its execution in the minimum time possible. We will study Time Complexity in detail in later sections.

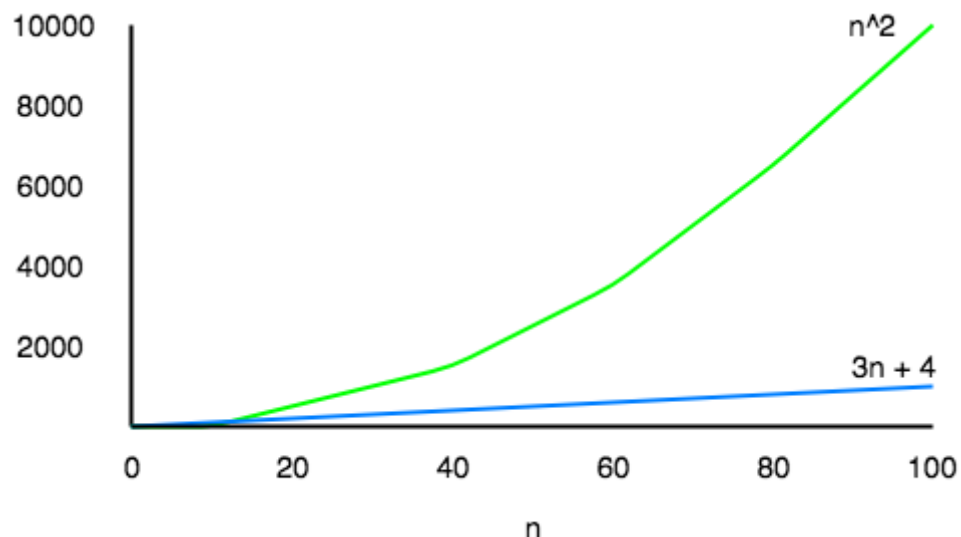
Asymptotic Notation

When it comes to analyzing the complexity of any algorithm in terms of time and space, we can never provide an exact number to define the time and space required by the algorithm. So instead, we express it using some standard notations, also known as Asymptotic Notations.

When we analyze an algorithm, we generally get a formula to represent the amount of time required for execution or the time required by the computer to run the lines of code of the algorithm, number of memory accesses, number of comparisons, temporary variables occupying memory space, etc. Unfortunately,

this formula often contains unimportant details that don't tell us anything about the running time.

For example, if some algorithm has a time complexity of $T(n) = (n^2 + 3n + 4)$, which is a quadratic equation. Then, for large values of n , the $3n + 4$ part will become insignificant compared to the n^2 part.



For $n = 1000$, n^2 will be 1000000 while $3n + 4$ will be 3004.

Also, When we compare the execution times of two algorithms, the constant coefficients of higher-order terms are neglected.

An algorithm that takes a time of $200n^2$ will be faster than some other algorithm that takes n^3 time for any value of n larger than 200. Since we're only interested in the asymptotic behavior of the growth of the function, we can ignore the constant factor.

What is Asymptotic Behaviour

The word Asymptotic means approaching a value or curve arbitrarily closely (i.e., as some sort of limit is taken).

Remember studying about Limits in High School. This is the same.

The only difference being, here we do not have to find the value of any expression where n is approaching any finite number or infinity. Still, in the case of Asymptotic notations, we use the same model to ignore the constant factors and insignificant parts of an expression better to represent the complexities of

algorithms in a single coefficient. Thus, we can make a comparison between algorithms easily.

Let's take an example to understand this:

If we have two algorithms with the following expressions representing the time required by them for execution, then:

$$\text{Expression 1: } (20n^2 + 3n - 4)$$

$$\text{Expression 2: } (n^3 + 100n - 2)$$

As per asymptotic notations, we should just worry about how the function will grow as the value of n (input) will grow, entirely depending on n^2 for Expression 1 and n^3 for Expression 2. Hence, we can clearly say that the algorithm for which running time is represented by Expression 2 will grow faster than the other one, simply by analyzing the highest power coefficient and ignoring the other constants(20 in $20n^2$) and insignificant parts of the expression($3n - 4$ and $100n - 2$).

The main idea behind casting aside the less important part is to make things manageable.

We need to analyze the algorithm to find an expression to define its time requirements and then analyze how that expression will grow as the input(n) grows.

Types of Asymptotic Notations

We use three types of asymptotic notations to represent the growth of any algorithm as input increases:

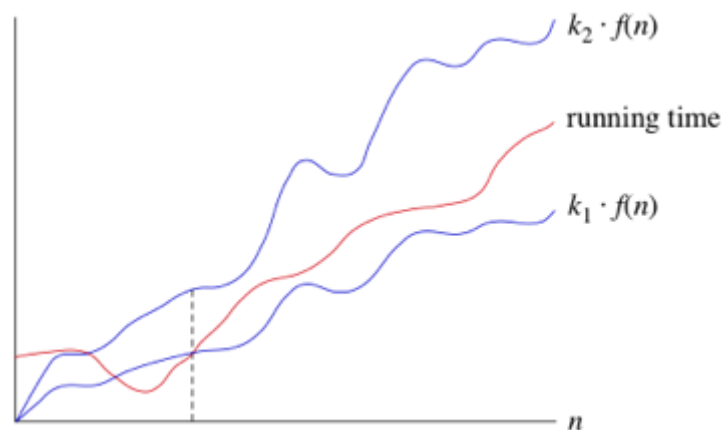
1. Big Theta (Θ)
2. Big Oh(O)
3. Big Omega (Ω)

Tight Bounds: Theta

When we say tight bounds, we mean that the time complexity represented by the Big- Θ notation is like the average value or range within which the actual time of execution of the algorithm will be.

For example, if for some algorithm the time complexity is represented by the expression $3n^2 + 5n$, and we use the Big- Θ notation to represent this, then the time complexity would be $\Theta(n^2)$, ignoring the constant-coefficient and removing the insignificant part, which is $5n$.

Here, in the example above, the complexity of $\Theta(n^2)$ means that the average time for any input n will remain in between, $k_1 \cdot n^2$ and $k_2 \cdot n^2$, where k_1, k_2 are two constants, thereby tightly binding the expression representing the growth of the algorithm.



Upper Bounds: Big-O

This notation is known as the upper bound of the algorithm or the Worst Case of an algorithm.

It tells us that a certain function will never exceed a specified time for any value of input n .

The question is why we need this representation when we already have the big- Θ notation, which represents the tightly bound running time for any algorithm. Let's take a small example to understand this.

Consider a Linear Search algorithm, in which we traverse an array of elements, one by one, to search a given number.

In the Worst case, starting from the front of the array, we find the element or number we are searching for at the end, which will lead to a time complexity of n , where n represents the number of total elements.

But the element that we are searching for maybe the first element of the array, in which case the time complexity will be 1.

Now, in this case, saying that the big- Θ or tight bound time complexity for Linear search is $\Theta(n)$, will mean that the time required will always be related to n , as this is the right way to represent the average time complexity, but when we use the big- O notation, we mean to say that the time complexity is $O(n)$, which means that the time complexity will never exceed n , defining the upper bound, hence saying that it can be less than or equal to n , which is the correct representation.

This is why you will see Big- O notation is used to represent the time complexity of an algorithm because it makes more sense.

Lower Bounds: Omega

Big Omega notation is used to define the lower bound of any algorithm, or we can say the best case of any algorithm.

This always indicates the minimum time required for any algorithm for all input values, therefore the best case of any algorithm.

In simple words, when we represent a time complexity for any algorithm in the form of big- Ω , we mean that the algorithm will take at least this much time to complete its execution. But, of course, it can take more time than this too.

Mathematical Concepts

Armstrong Number

Given a number x , determine whether the given number is Armstrong's number or not. For example, a positive integer of **n digits** is called an Armstrong number of **order n** (order is several digits).

$$abcd\dots = \text{pow}(a,n) + \text{pow}(b,n) + \text{pow}(c,n) + \text{pow}(d,n) + \dots$$

Example:

Input : 153

Output : Yes

153 is an Armstrong number.

$$1*1*1 + 5*5*5 + 3*3*3 = 153$$

Input : 120

Output : No

120 is not a Armstrong number.

$$1*1*1 + 2*2*2 + 0*0*0 = 9$$

Input : 1253

Output : No

1253 is not a Armstrong Number

$$1*1*1*1 + 2*2*2*2 + 5*5*5*5 + 3*3*3*3 = 723$$

Input : 1634

Output : Yes

$$1*1*1*1 + 6*6*6*6 + 3*3*3*3 + 4*4*4*4 = 1634$$

Roman numeral to integers

Given a Roman numeral, the task is to find its corresponding decimal value.

Input: IX

Output: 9

IX is a Roman symbol that represents 9

Input: XL

Output: 40

XL is a Roman symbol which represents 40

Input: MCMIV

Output: 1904

M is a thousand,

CM is nine hundred and

IV is four

SYMBOL	VALUE
I	1
IV	4
V	5
IX	9
X	10
XL	40
L	50
XC	90
C	100
CD	400
D	500
CM	900
M	1000

Approach: A number in Roman Numerals is a string of these symbols written in descending order(e.g., M's first, followed by D's, etc.). However, in a few specific cases, to avoid four characters being repeated in succession(such as IIII or XXXX), **subtractive notation** is often used as follows:

- **I** placed before **V** or **X** indicates one less, so four is **IV** (one less than 5) and 9 is **IX** (one less than 10).
- **X** placed before **L** or **C** indicates ten less, so forty is **XL** (10 less than 50), and 90 is **XC** (ten less than a hundred).

- **C** placed before **D** or **M** indicates a hundred less, so four hundred is **CD** (a hundred less than five hundred), and nine hundred is **CM** (a hundred less than a thousand).

Algorithm to convert Roman Numerals to Integer Number:

1. Split the Roman Numeral string into Roman Symbols (character).
2. Then, convert each symbol of Roman Numerals into the value it represents.
3. Take symbols one by one starting from index 0:
 - a. If the current value of the symbol is greater than or equal to the value of the next symbol, then add this value to the running total.
 - b. Else subtract this value by adding the value of the next symbol to the running total.

Trailing zeros in factorial

Given an integer n , write a function that returns the count of trailing zeroes in $n!$.

Examples :

```
Input: n = 5
Output: 1
Factorial of 5 is 120 which has one trailing 0.

Input: n = 20
Output: 4
Factorial of 20 is 2432902008176640000, which has
Four trailing zeroes.
```

```
Input: n = 100
```

```
Output: 24
```

1. **Approach:**

A simple method is first to calculate factorial of n , then count trailing 0s in the result (We can count trailing 0s by repeatedly dividing the factorial by ten till the remainder is 0).

2. The above method can cause overflow for slightly bigger numbers as the factorial of a number is a big number (See factorial of 20 given in above examples). The idea is to consider prime factors of a factorial n . A trailing zero is always produced by prime factors 2 and 5. If we can count the number of 5s and 2s, our task is done. Consider the following examples.

$n = 5$: There is one 5 and 3 2s in prime factors of $5!$ ($2 * 2 * 2 * 3 * 5$). So a count of trailing 0s is 1.

$n = 11$: There are two 5s and eight 2s in prime factors of $11!$ ($2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 3 * 5 * 5 * 7$). So the count of trailing 0s is 2.

3. We can easily observe that the number of 2s in prime factors is always more than or equal to 5s. So if we count 5s in prime factors, we are done. *How to count the total number of 5s in prime factors of $n!$?* A simple way is to calculate $\text{floor}(n/5)$. For example, $7!$ has one 5, $10!$ has two 5s. It is not done yet. There is one more thing to consider. Numbers like 25, 125, etc., have more than one 5. For example, if we

consider 28! we get one extra 5, and the number of 0s becomes 6.

Handling this is simple, first, divide n by five and remove all single 5s, then divide by 25 to remove extra 5s, and so on. Following is the summarized formula for counting trailing 0s.

```
Trailing 0s in n! = count of 5s in prime factors of n!  
                  = floor(n/5) + floor(n/25) + floor(n/125) +  
                  . . . .
```

Palindrome numbers

Palindrome number in c: A **palindrome number** is *a number that is the same after reverse*. For example, 121, 34543, 343, 131, 48984 are the palindrome numbers.

Palindrome number algorithm

- Get the number from the user
- Hold the number in the temporary variable
- Reverse the number
- Compare the temporary number with the reversed number
- If both numbers are the same, print the palindrome number
- Else print not a palindrome number

Factors, HCF, LCM

Factors and Multiples: All the numbers that divide a number completely, i.e., without leaving any remainder, are called factors of that number. For example, 24 is completely divisible by 1, 2, 3, 4, 6, 8, 12, 24. Therefore, each of these numbers is called a factor of 24, and 24 is called a multiple of each of these numbers.

LCM: The least number divisible by each given number is called the least common multiple of those numbers. For example, consider the numbers 3, 31, and 62 (2×31). The LCM of these numbers would be $2 \times 3 \times 31 = 186$.

To find the LCM of the given numbers, we express each number as the product of prime numbers. The product highest power of the prime numbers that appear in the prime factorization of any of the numbers gives us the LCM.

For example, consider the numbers 2, 3, 4 (2×2), 5, 6 (2×3). The LCM of these numbers is $2 \times 2 \times 3 \times 5 = 60$. The highest power of 2 comes from the prime factorization of 4. The highest power of 3 comes from the prime factorization of 3 and the prime factorization of 6. The highest power of 5 comes from the prime factorization of 5.

HCF: The largest number that divides two or more numbers is the highest common factor (HCF) for those numbers. For example, consider the numbers 30 ($2 \times 3 \times 5$), 36 ($2 \times 2 \times 3 \times 3$), 42 ($2 \times 3 \times 7$), 45 ($3 \times 3 \times 5$). 3 is the largest number that divides each of these numbers, and hence, is the HCF for these numbers.

HCF is also known as Greatest Common Divisor (GCD).

$$\begin{array}{r}
 30 \overline{) 42} \quad 1 \\
 \underline{- 30} \\
 12 \overline{) 30} \quad 2 \\
 \underline{- 24} \\
 6 \overline{) 12} \quad 2 \\
 \underline{- 12} \\
 0
 \end{array}$$

HCF

To find the HCF of two or more numbers, express each number as a product of prime numbers. The product of the least powers of common prime terms gives us the HCF. This is the method we illustrated in the above step.

Also, for finding the HCF of two numbers, we can proceed by the long division method. First, we divide the larger number by the smaller number (divisor).

Now, we divide the divisor by the remainder obtained in the previous station, we. Finally, we repeat the same procedure until we get zero as the remainder.

At that stage, the last divisor would be the required HCF.

For two numbers 'a' and 'b' **LCM x HCF = a x b**

Prime Numbers

Given a positive integer N. The task is to write a C program to check if the number is **prime or not**.

The idea to solve this problem is to iterate through all the numbers starting from 2 to $\sqrt{N}$ using a for loop, and for every number, check if it divides N. If we find any number that divides, we return false. On the other hand, if we did not find any number between 2 and $\sqrt{N}$ that divides N, N is prime, and we will return True.

Why did we choose $\text{sqrt}(N)$?

The reason is that the smallest and greater than one factor of a number cannot be more than the sqrt of N . And we stop as soon as we find a factor. For example, if N is 49, the smallest factor is 7. For 15, the smallest factor is 3.

Prime sieve / Sieve of Eratosthenes

The sieve of Eratosthenes is one of the most efficient ways to find all primes smaller than n when n is smaller than 10 million or so (Ref [Wiki](#)).

Following is the algorithm to find all the prime numbers less than or equal to a given integer n by the Eratosthenes method:

When the algorithm terminates, all the numbers in the list that are not marked are prime.

Explanation with Example:

Let us take an example when $n = 50$. So we need to print all prime numbers smaller than or equal to 50.

We create a list of all numbers from 2 to 50.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

According to the algorithm, we will mark all the numbers divisible by two and are greater than or equal to their square.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

Now we move to our next unmarked number, 3, and mark all the multiples of 3 and are greater than or equal to its square.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

We move to our next unmarked number, 5, and mark all multiples of 5 and are greater than or equal to its square.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

We continue this process, and our final table will look like below:

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

So the prime numbers are the unmarked ones: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47.

Bits and Bytes

Working on bytes, or data types comprising of bytes like ints, floats, doubles or even data structures which stores large amount of bytes is normal for a programmer. In some cases, a programmer needs to go beyond this - that is to say that in a deeper level where the importance of bits is realized.

Operations with bits are used in Data compression (data is compressed by converting it from one representation to another, to reduce the space) ,Exclusive-Or Encryption (an algorithm to encrypt the data for safety issues). In order to encode, decode or compress files we have to extract the data at bit level. Bitwise Operations are faster and closer to the system and sometimes optimize the program to a good level.

We all know that 1 byte comprises of 8 bits and any integer or character can be represented using bits in computers, which we call its binary form(contains only 1 or 0) or in its base 2 form.

Example:

$$1) 14 = \{1110\}_2$$

$$= 1 * 2^3 + 1 * 2^2 + 1 * 2^1 + 0 * 2^0$$

$$= 14.$$

$$2) 20 = \{10100\}_2$$

$$= 1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 0 * 2^1 + 0 * 2^0$$

$$= 20.$$

For characters, we use ASCII representation, which are in the form of integers which again can be represented using bits as explained above.

Bitwise operators

There are different bitwise operations used in the bit manipulation. These bit operations operate on the individual bits of the bit patterns. Bit operations are fast and can be used in optimizing time complexity. Some common bit operators are:

NOT (~): Bitwise NOT is an unary operator that flips the bits of the number i.e., if the i th bit is 0, it will change it to 1 and vice versa. Bitwise NOT is nothing but simply the one's complement of a number. Lets take an example.

$$N = 5 = (101)_2$$

$$\sim N = \sim 5 = \sim (101)_2 = (010)_2 = 2$$

AND (&): Bitwise AND is a binary operator that operates on two equal-length bit patterns. If both bits in the compared position of the bit patterns are 1, the bit in the resulting bit pattern is 1, otherwise 0.

$$A = 5 = (101)_2, B = 3 = (011)_2 \quad A \& B = (101)_2 \& (011)_2 = (001)_2 = 1$$

OR (|): Bitwise OR is also a binary operator that operates on two equal-length bit patterns, similar to bitwise AND. If both bits in the compared position of the bit patterns are 0, the bit in the resulting bit pattern is 0, otherwise 1.

$$A = 5 = (101)_2, B = 3 = (011)_2$$

$$A | B = (101)_2 | (011)_2 = (111)_2 = 7$$

XOR (^): Bitwise XOR also takes two equal-length bit patterns. If both bits in the compared position of the bit patterns are 0 or 1, the bit in the resulting bit pattern is 0, otherwise 1.

$$A = 5 = (101)_2, B = 3 = (011)_2$$

$$A \wedge B = (101)_2 \wedge (011)_2 = (110)_2 = 6$$

Left Shift (<<): Left shift operator is a binary operator which shift the some number of bits, in the given bit pattern, to the left and append 0 at the end. Left shift is equivalent to multiplying the bit pattern with

2^k

(if we are shifting k bits).

$$1 \ll 1 = 2 = 2^1$$

$$1 \ll 2 = 4 = 2^2 \quad 1 \ll 3 = 8 = 2^3$$

$$1 \ll 4 = 16 = 2^4$$

...

$$1 \ll n = 2^n$$

Right Shift (>>): Right shift operator is a binary operator which shift the some number of bits, in the given bit pattern, to the right and append 1 at the end.

Right shift is equivalent to dividing the bit pattern with 2^k (if we are shifting k bits).

$$4 \gg 1 = 2$$

$$6 \gg 1 = 3$$

$$5 \gg 1 = 2$$

$$16 \gg 4 = 1$$

X	Y	X&Y	X Y	X^Y	~(X)
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Bitwise operators are good for saving space and sometimes to cleverly remove dependencies.

Note: All left and right side taken in this article, are taken with reference to the reader.

Lets discuss some algorithms based on bitwise operations:

1) **How to check if a given number is a power of 2 ?**

Consider a number N and you need to find if N is a power of 2. Simple solution to this problem is to repeated divide N by 2 if N is even. If we end up with a 1 then N is power of 2, otherwise not. There are a special case also. If N = 0 then it is not a power of 2. Let's code it.

Above function will return true if x is a power of 2, otherwise false.

Time complexity of the above code is $O(\log N)$.

The same problem can be solved using bit manipulation. Consider a number x that we need to check for being a power for 2. Now think about the binary

representation of $(x-1)$. $(x-1)$ will have all the bits same as x , except for the rightmost 1 in x and all the bits to the right of the rightmost 1.

$$\text{Let, } x = 4 = (100)_2$$

$$x - 1 = 3 = (011)_2$$

$$\text{Let, } x = 6 = (110)_2$$

$$x - 1 = 5 = (101)_2$$

It might not seem obvious with these examples, but binary representation of $(x-1)$ can be obtained by simply flipping all the bits to the right of rightmost 1 in x and also including the rightmost 1.

Now think about $x \& (x-1)$. $x \& (x-1)$ will have all the bits equal to the x except for the rightmost 1 in x .

$$\text{Let, } x = 4 = (100)_2$$

$$x - 1 = 3 = (011)_2$$

$$x \& (x-1) = 4 \& 3 = (100)_2 \& (011)_2 = (000)_2$$

$$\text{Let, } x = 6 = (110)_2$$

$$x - 1 = 5 = (101)_2$$

$$x \& (x-1) = 6 \& 5 = (110)_2 \& (101)_2 = (100)_2$$

Properties for numbers which are powers of 2, is that they have one and only one bit set in their binary representation. If the number is neither zero nor a power of two, it will have 1 in more than one place. So if x is a power of 2 then $x \& (x-1)$ will be 0.

2) Count the number of ones in the binary representation of the given number.

The basic approach to evaluate the binary form of a number is to traverse on it and count the number of ones. But this approach takes $\log_2 N$ of time in every case.

Why $\log_2 N$?

As to get a number in its binary form, we have to divide it by 2, until it gets 0, which will take $\log_2 N$ of time.

With bitwise operations, we can use an algorithm whose running time depends on the number of ones present in the binary form of the given number. This algorithm is much better, as it will reach to $\log N$, only in its worst case.

Why this algorithm works ?

As explained in the previous algorithm, the relationship between the bits of x and $x-1$. So as in $x-1$, the rightmost 1 and bits right to it are flipped, then by performing $x \& (x-1)$, and storing it in x , will reduce x to a number containing number of ones (in its binary form) less than the previous state of x , thus increasing the value of count in each iteration.

Example:

$$n = 23 = \{10111\}_2 .$$

1. Initially, count = 0.

2. Now, n will change to $n \& (n-1)$. As $n-1 = 22 = \{10110\}_2$, then $n \& (n-1)$ will be $\{10111\}_2 \& \{10110\}_2$, which will be $\{10110\}_2$ which is equal to 22. Therefore n will change to 22 and count to 1.

3. As $n-1 = 21 = \{10101\}_2$, then $n \& (n-1)$ will be $\{10110\}_2 \& \{10101\}_2$, which will be $\{10100\}_2$ which is equal to 20. Therefore n will change to 20 and count to 2.

4. As $n-1 = 19 = \{10011\}_2$, then $n \& (n-1)$ will be $\{10100\}_2 \& \{10011\}_2$, which will be $\{10000\}_2$ which is equal to 16. Therefore n will change to 16 and count to 3.

5. As $n-1 = 15 = \{01111\}_2$, then $n \& (n-1)$ will be $\{10000\}_2 \& \{01111\}_2$, which will be $\{00000\}_2$ which is equal to 0. Therefore n will change to 0 and count to 4.

6. As $n = 0$, the the loop will terminate and gives the result as 4.

Complexity: $O(K)$, where K is the number of ones present in the binary form of the given number.

3) Check if the i^{th} bit is set in the binary form of the given number.

To check if the i^{th} bit is set or not (1 or not), we can use AND operator. How?

Let's say we have a number N , and to check whether it's i^{th} bit is set or not, we can AND it with the number 2^i . The binary form of 2^i contains only i^{th} bit as set (or 1), else every bit is 0 there. When we will AND it with N , and if the i^{th} bit of N is set, then it will return a non zero number (2^i to be specific), else 0 will be returned.

Using Left shift operator, we can write 2^i as $1 \ll i$. Therefore:

Example:

Let's say $N = 20 = \{10100\}_2$. Now let's check if its 2nd bit is set or not (starting from 0). For that, we have to AND it with $2^2 = 1 \ll 2 = \{100\}_2$.

$\{10100\} \& \{100\} = \{100\} = 2^2 = 4$ (non-zero number), which means its 2nd bit is set.

4) How to generate all the possible subsets of a set ?

A big advantage of bit manipulation is that it can help to iterate over all the subsets of an N-element set. As we all know there are 2^N possible subsets of any given set with N elements. What if we represent each element in a subset with a bit. A bit can be either 0 or 1, thus we can use this to denote whether the corresponding element belongs to this given subset or not. So each bit pattern will represent a subset.

Consider a set A of 3 elements.

$A = \{a, b, c\}$

Now, we need 3 bits, one bit for each element. 1 represent that the corresponding element is present in the subset, whereas 0 represent the corresponding element is not in the subset. Let's write all the possible combination of these 3 bits.

$$0 = (000)_2 = \{\}$$

$$1 = (001)_2 = \{c\}$$

$$2 = (010)_2 = \{b\}$$

$$3 = (011)_2 = \{b, c\}$$

$$4 = (100)_2 = \{a\}$$

$$5 = (101)_2 = \{a, c\}$$

$$6 = (110)_2 = \{a, b\}$$

$$7 = (111)_2 = \{a, b, c\}$$

Pseudo Code:

```
possibleSubsets(A, N):
    for i = 0 to 2^N:
        for j = 0 to N:
            if jth bit is set in i:
                print A[j]
        print '\n'
```

5) Find the largest power of 2 (most significant bit in binary form), which is less than or equal to the given number N.

Idea: Change all the bits which are at the right side of the most significant digit, to 1.

Property: As we know that when all the bits of a number N are 1, then N must be equal to the $2^i - 1$, where i is the number of bits in N.

Example:

Let's say binary form of a N is $\{1111\}_2$ which is equal to 15.

$15 = 2^4 - 1$, where 4 is the number of bits in N.

This property can be used to find the largest power of 2 less than or equal to N.
How?

If we somehow, change all the bits which are at right side of the most significant bit of N to 1, then the number will become $x + (x-1) = 2 * x - 1$, where x is the required answer.

Example:

Let's say $N = 21 = \{10101\}$, here most significant bit is the 4th one. (counting from 0th digit) and so the answer should be 16.

So let's change all the right side bits of the most significant bit to 1. Now the number changes to

$\{11111\} = 31 = 2 * 16 - 1 = Y$ (let's say).

Now the required answer is $(Y+1) \gg 1$ or $(Y+1)/2$.

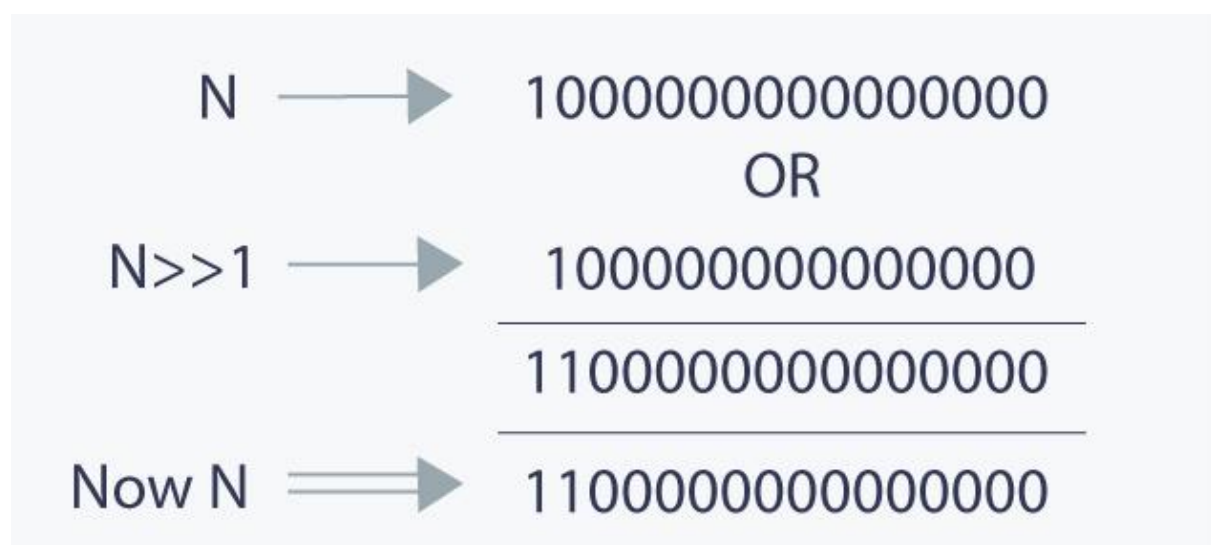
Now the question arises here is how can we change all right side bits of most significant bit to 1?

Let's take the N as 16 bit integer and binary form of N is $\{1000000000000000\}$.

Here we have to change all the right side bits to 1.

Initially we will copy that most significant bit to its adjacent right side by:

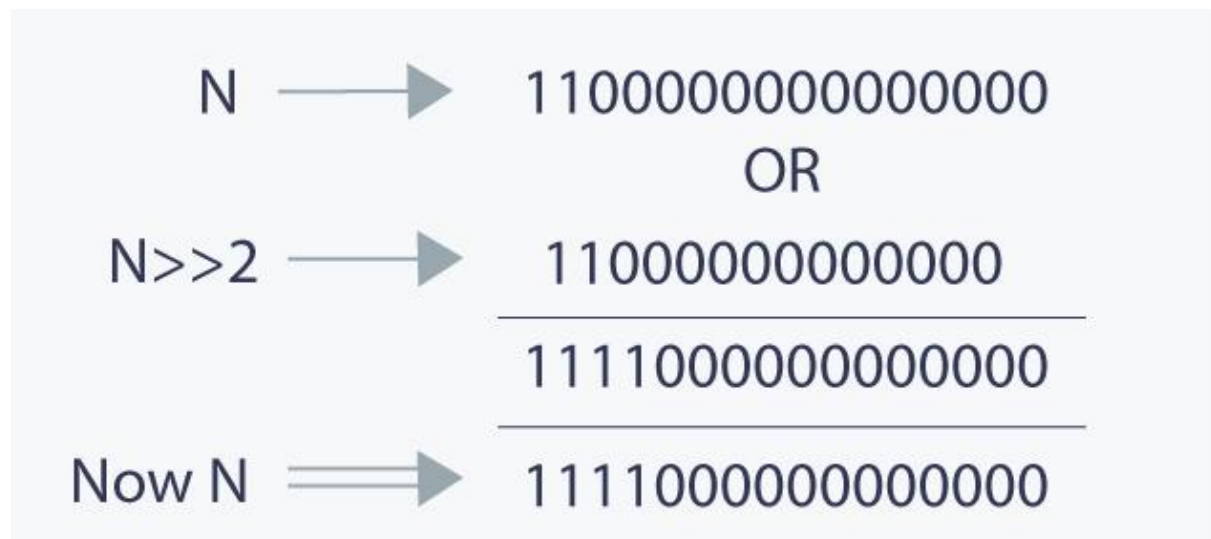
$N = N | (N \gg 1)$.



As you can see, in above diagram, after performing the operation, rightmost bit has been copied to its adjacent place.

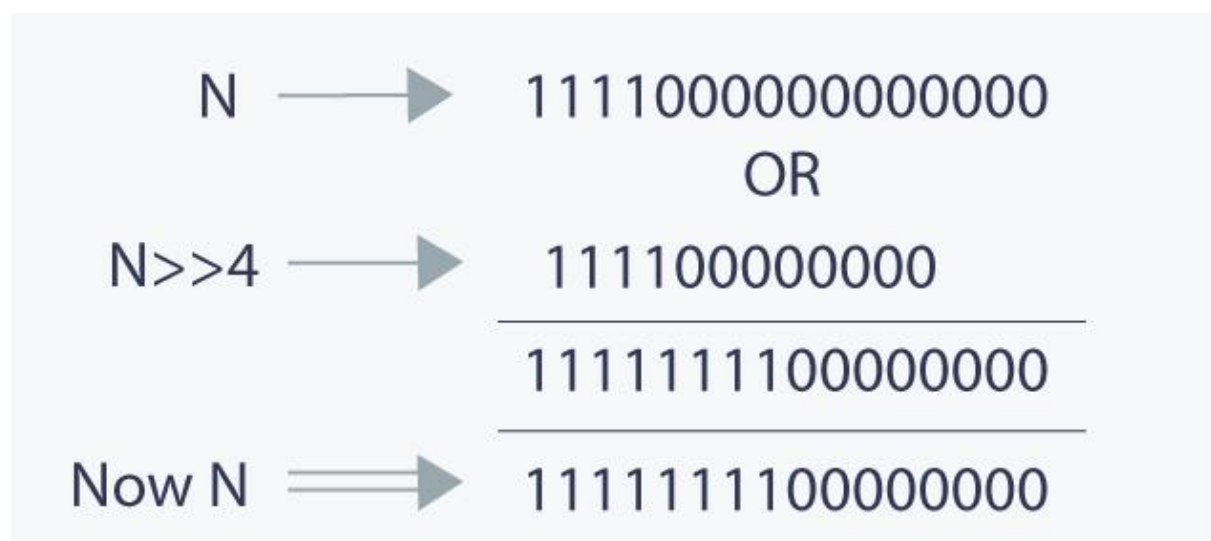
Now we will copy the 2 rightmost set bits to their adjacent right side.

$N = N | (N \gg 2).$



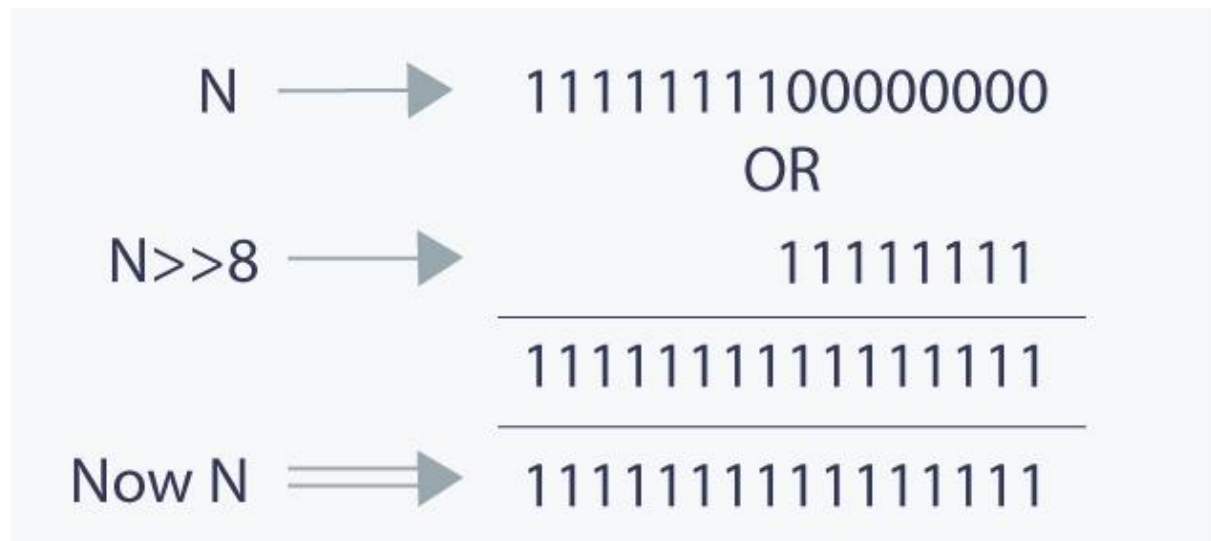
Now we will copy the 4 rightmost set bit to their adjacent right side.

$N = N | (N \gg 4)$



Now we will copy these 8 rightmost set bits to their adjacent right side.

$N = N | (N \gg 8)$



Now all the right side bits of the most significant set bit has been changed to 1. This is how we can change right side bits. This explanation is for 16 bit integer, and it can be extended for 32 or 64 bit integer too.

Implementation:

```
def largest_power(N):
```

```
    //changing all right side bits to 1.
```

```
    N = N | (N >> 1);
```

```
    N = N | (N >> 2);
```

```
    N = N | (N >> 4);
```

```
    N = N | (N >> 8);
```

```
    //as now the number is 2 * x-1, where x is required answer, so adding 1 and  
    dividing it by
```

```
    2.
```

```
    return (N+1) >> 1
```

Tricks with Bits:

1) $x \wedge (x \& (x-1))$: Returns the rightmost 1 in binary representation of x.

As explained above, $(x \& (x - 1))$ will have all the bits equal to the x except for the rightmost 1 in x. So if we do bitwise XOR of x and $(x \& (x-1))$, it will simply return the rightmost 1. Let's see an example.

$$x = 10 = (1010)_2 \quad x \& (x-1) = (1010)_2 \& (1001)_2 = (1000)_2$$

$$x \wedge (x \& (x-1)) = (1010)_2 \wedge (1000)_2 = (0010)_2$$

2) $x \& (-x)$: Returns the rightmost 1 in binary representation of x

$(-x)$ is the two's complement of x. $(-x)$ will be equal to one's complement of x plus 1.

Therefore $(-x)$ will have all the bits flipped that are on the left of the rightmost 1 in x. So $x \& (-x)$ will return rightmost 1.

$$x = 10 = (1010)_2$$

$$(-x) = -10 = (0110)_2$$

$$x \& (-x) = (1010)_2 \& (0110)_2 = (0010)_2$$

3) $x \mid (1 \ll n)$: Returns the number x with the nth bit set.

$(1 \ll n)$ will return a number with only nth bit set. So if we OR it with x it will set the nth bit of x.

$$x = 10 = (1010)_2 \quad n = 2$$

$$1 \ll n = (0100)_2$$

$$x \mid (1 \ll n) = (1010)_2 \mid (0100)_2 = (1110)_2$$

Applications of bit operations:

- 1) They are widely used in areas of graphics ,specially XOR(Exclusive OR) operations.
- 2) They are widely used in the embedded systems, in situations, where we need to set/clear/toggle just one single bit of a specific register without modifying the other contents. We can do OR/AND/XOR operations with the appropriate mask for the bit position.
- 3) Data structure like n-bit map can be used to allocate n-size resource pool to represent the current status.
- 4) Bits are used in networking, framing the packets of numerous bits which is sent to another system generally through any type of serial interface.

To see a creative use of Bitwise operators, you can refer [this amazing article](#) , where the bit wise operations are smartly used while developing an online calendar for events.

Recursion

In this tutorial, you will learn to create a recursive function (a function that calls itself).

What is recursion?

Recursion is the process of defining something in terms of itself.

A physical world example would be to place two parallel mirrors facing each other. Any object in between them would be reflected recursively.

Python Recursive Function

In Python, we know that a function can call other functions. It is even possible for the function to call itself. These types of construct are termed as recursive functions.

The following image shows the working of a recursive function called recurse.

Python Recursive Function

Recursive Function in Python

Following is an example of a recursive function to find the factorial of an integer.

Factorial of a number is the product of all the integers from 1 to that number. For example, the factorial of 6 (denoted as 6!) is $1*2*3*4*5*6 = 720$.

Example of a recursive function

```
def factorial(x):  
    """This is a recursive function  
    to find the factorial of an integer"""  
  
    if x == 1:  
        return 1  
    else:  
        return (x * factorial(x-1))  
  
num = 3  
print("The factorial of", num, "is", factorial(num))  
Output
```

The factorial of 3 is 6

In the above example, factorial() is a recursive function as it calls itself.

When we call this function with a positive integer, it will recursively call itself by decreasing the number.

Each function multiplies the number with the factorial of the number below it until it is equal to one. This recursive call can be explained in the following steps.

```
factorial(3)      # 1st call with 3  
3 * factorial(2)  # 2nd call with 2
```

```
3 * 2 * factorial(1) # 3rd call with 1
3 * 2 * 1           # return from 3rd call as number=1
3 * 2               # return from 2nd call
6                   # return from 1st call
```

Let's look at an image that shows a step-by-step process of what is going on:

Factorial by a recursive method

Working of a recursive factorial function

Our recursion ends when the number reduces to 1. This is called the base condition.

Every recursive function must have a base condition that stops the recursion or else the function calls itself infinitely.

The Python interpreter limits the depths of recursion to help avoid infinite recursions, resulting in stack overflows.

By default, the maximum depth of recursion is 1000. If the limit is crossed, it results in RecursionError. Let's look at one such condition.

```
def recursor():
```

```
    recursor()
```

```
recursor()
```

Output

Traceback (most recent call last):

File "<string>", line 3, in <module>

File "<string>", line 2, in a

File "<string>", line 2, in a

File "<string>", line 2, in a

[Previous line repeated 996 more times]

RecursionError: maximum recursion depth exceeded

Advantages of Recursion

Recursive functions make the code look clean and elegant.

A complex task can be broken down into simpler sub-problems using recursion.

Sequence generation is easier with recursion than using some nested iteration.

Disadvantages of Recursion

Sometimes the logic behind recursion is hard to follow through.

Recursive calls are expensive (inefficient) as they take up a lot of memory and time.

Recursive functions are hard to debug.

Array

An array is defined as a collection of items that are stored at contiguous memory locations. It is a container which can hold a fixed number of items, and these items should be of the same type. An array is popular in most programming languages like C/C++, JavaScript, etc.

Array is an idea of storing multiple items of the same type together and it makes easier to calculate the position of each element by simply adding an offset to the base value. A combination of the arrays could save a lot of time by reducing the overall size of the code. It is used to store multiple values in single variable. If you have a list of items that are stored in their corresponding variables like this:

```
car1 = "Lamborghini"
```

```
car2 = "Bugatti"
```

```
car3 = "Koenigsegg"
```

If you want to loop through cars and find a specific one, you can use the array.

The array can be handled in Python by a module named **array**. It is useful when we have to manipulate only specific data values. Following are the terms to understand the concept of an array:

Element - Each item stored in an array is called an element.

Index - The location of an element in an array has a numerical index, which is used to identify the position of the element.

Array Representation

An array can be declared in various ways and different languages. The important points that should be considered are as follows:

- Index starts with 0.
- We can access each element via its index.
- The length of the array defines the capacity to store the elements.

Array operations

Some of the basic operations supported by an array are as follows:

- **Traverse** - It prints all the elements one by one.
- **Insertion** - It adds an element at the given index.
- **Deletion** - It deletes an element at the given index.
- **Search** - It searches an element using the given index or by the value.
- **Update** - It updates an element at the given index.

The Array can be created in Python by importing the array module to the python program.

1. from array **import** *
2. arrayName = array(typecode, [initializers])

Accessing array elements

We can access the array elements using the respective indices of those elements.

1. **import** array as arr
2. a = arr.array('i', [2, 4, 6, 8])
3. print("First element:", a[0])
4. print("Second element:", a[1])
5. print("Second last element:", a[-1])

Output:

First element: 2

Second element: 4

Second last element: 8

Explanation: In the above example, we have imported an array, defined a variable named as "a" that holds the elements of an array and print the elements by accessing elements through indices of an array.

How to change or add elements

Arrays are mutable, and their elements can be changed in a similar way like lists.

1. `import array as arr`
2. `numbers = arr.array('i', [1, 2, 3, 5, 7, 10])`
- 3.
4. `# changing first element`
5. `numbers[0] = 0`
6. `print(numbers) # Output: array('i', [0, 2, 3, 5, 7, 10])`
- 7.
8. `# changing 3rd to 5th element`
9. `numbers[2:5] = arr.array('i', [4, 6, 8])`
10. `print(numbers) # Output: array('i', [0, 2, 4, 6, 8, 10])`

Output:

```
array('i', [0, 2, 3, 5, 7, 10])
```

```
array('i', [0, 2, 4, 6, 8, 10])
```

Explanation: In the above example, we have imported an array and defined a variable named as "numbers" which holds the value of an array. If we want to change or add the elements in an array, we can do it by defining the particular index of an array where you want to change or add the elements.

Why to use arrays in Python?

A combination of arrays saves a lot of time. The array can reduce the overall size of the code.

How to delete elements from an array?

The elements can be deleted from an array using Python's **del** statement. If we want to delete any value from the array, we can do that by using the indices of a particular element.

1. **import** array as arr
2. number = arr.array('i', [1, 2, 3, 3, 4])
3. del number[2] # removing third element
4. print(number) # Output: array('i', [1, 2, 3, 4])

Output:

```
array('i', [10, 20, 40, 60])
```

Explanation: In the above example, we have imported an array and defined a variable named as "number" which stores the values of an array. Here, by using del statement, we are removing the third element [3] of the given array.

Finding the length of an array

The length of an array is defined as the number of elements present in an array. It returns an integer value that is equal to the total number of the elements present in that array.

Syntax

1. len(array_name)

Array Concatenation

We can easily concatenate any two arrays using the + symbol.

Example

1. a=arr.array('d',[1.1 , 2.1 ,3.1,2.6,7.8])
2. b=arr.array('d',[3.7,8.6])

3. `c=arr.array('d')`
4. `c=a+b`
5. `print("Array c = ",c)`

Output:

Array c= array('d', [1.1, 2.1, 3.1, 2.6, 7.8, 3.7, 8.6])

Explanation

In the above example, we have defined variables named as "a, b, c" that hold the values of an array.

Example

1. `import array as arr`
2. `x = arr.array('i', [4, 7, 19, 22])`
3. `print("First element:", x[0])`
4. `print("Second element:", x[1])`
5. `print("Second last element:", x[-1])`

Output:

First element: 4

Second element: 7

Second last element: 22

Explanation: In the above example, first, we have imported an array and defined a variable named as "x" which holds the value of an array and then, we have printed the elements using the indices of an array.